Personalized Content Discovery

A Recommendation System on the Netflix Prize Dataset

Technical Report • Collaborative Filtering & Matrix Factorization

100.5M	6.07M	20,000	3,755	0.838	0.059
RATINGS (FULL SET)	RATINGS USED	USERS	MOVIES	BEST RMSE (SVD)	BEST MAP@10 (SVD)

1. Problem Understanding

Streaming platforms live or die by content discovery: with tens of thousands of titles, users can only engage with what the system surfaces. We address the classic **Netflix Prize** task — learning user preferences from historical 1–5 star ratings and using them to (a) **predict ratings** for unseen movies and (b) **generate ranked Top-K recommendations**. The two goals are related but distinct: a model can predict ratings accurately yet rank poorly, so we evaluate both an error metric (RMSE) and a ranking metric (MAP@10). The data is extremely sparse — each user rates a tiny fraction of the catalogue — which makes collaborative filtering and latent-factor models the natural toolset.

Objectives

- Learn preferences from the user×item rating matrix.
- Predict ratings for unseen (user, movie) pairs.
- Produce personalized, ranked Top-10 recommendations.
- Compare multiple approaches on accuracy, ranking quality, and cost.

2. Exploratory Data Analysis

We parsed the four `combined_data_*.txt` files plus `movie_titles.csv`. To keep computation tractable and signal dense, we filtered to users with >= 20 ratings and movies with >= 50 ratings, then sampled the 20,000 most active users — yielding a **6.07M-rating** working set across **3,755** movies. Even after filtering for activity the matrix is **91.9% sparse** (8.1% dense), confirming sparsity as the central modeling challenge.

Key statistics

- **6,073,039** ratings • 20,000 users • 3,755 movies
- Sparsity **91.9%** (density 8.1%)
- Global mean rating **3.44** / 5
- Ratings/user: median **267**, mean 304, max 3,733
- Ratings/movie: median **308**, mean 1,617, max 18,940
- Timespan: **1999-11-11** → **2005-12-31**

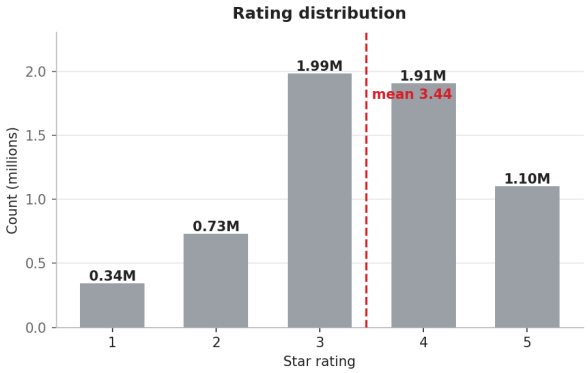

Figure 1. Ratings skew positive — 3- and 4-star dominate; the mean (3.44) sits above the midpoint.

Insights & implications

- **Positive skew.** Users mostly rate titles they chose to watch, so a relevance threshold of >= 3.5 stars (used for MAP@10) cleanly separates genuine likes from the long 1–3 tail.
- **Popularity is highly concentrated.** The top titles (Pirates of the Caribbean, The Sixth Sense, Silence of the Lambs, LOTR: Fellowship) each gather ~18-19k ratings, while the median movie has only 308 — a classic long-tail that drives popularity bias in naive models.

- **Strong per-user bias.** Heavy raters and lenient/harsh raters both exist, motivating explicit user/item bias terms before any interaction modeling.
- **Sparsity.** 91.9% empty cells mean memory-based neighbours are noisy; latent factors generalize better.

3. Methodology

3.1 Data processing & train/test split

Ratings are parsed into a tidy table (userId, movieId, rating, date) and remapped to contiguous integer indices. We use a **per-user temporal hold-out**: for each user, ratings are sorted by date and the most recent 20% are placed in the test set (with at least a few kept in train). This mimics real deployment — predicting the *future* from the *past* — and prevents leakage that a random split would introduce. Users with too few ratings keep all interactions in train.

3.2 Models

- **Baseline (bias model).** $\hat{r}(u,i) = \mu + b_u + b_i$ with regularized user/item biases. Captures rater leniency and movie quality; a strong, cheap reference.
- **Item-based CF.** Shrunk cosine similarity between items on bias-removed residuals; predict from a user's rated neighbours. Stable item-item structure, intuitive 'because you watched X'.
- **User-based CF.** Symmetric neighbour approach over users. Included mainly for comparison; costlier and noisier at scale because tastes drift and users are numerous.
- **SVD / matrix factorization.** Biased latent-factor model $\hat{r}(u,i)=\mu+b_u+b_i+p_u \cdot q_i$, trained with mini-batch SGD and L2 regularization (50 factors). Learns compact taste vectors that generalize across the sparse matrix — the family that won the Netflix Prize.

4. Evaluation (RMSE & MAP@10)

We evaluate against the two **mandatory** metrics — **RMSE** for rating-prediction accuracy and **MAP@10** for recommendation ranking quality — and report a set of optional metrics for context. Each required element of the protocol is described explicitly below.

- **Train–test split methodology.** We use a *per-user temporal hold-out*. For every user we sort their ratings by timestamp and place the most recent 20% in the test set and the earlier 80% in train, keeping at least 5 ratings in train (users with too few ratings keep all interactions in train). Splitting by time rather than randomly mimics real deployment — predicting *future* watches from *past* behaviour — and prevents the look-ahead leakage a random split would introduce.
- **Relevance definition.** A held-out test movie is counted as *relevant* if the user's actual rating is >= **3.5** stars, exactly as specified by the problem statement. Ratings below 3.5 are treated as non-relevant when scoring all ranking metrics.
- **Top-10 generation procedure.** For each evaluated user we (1) form the candidate pool from every movie the user did *not* rate in training, (2) score each candidate with the trained model (predicted rating for the bias and SVD models; weighted-neighbour score for the CF models), (3) sort candidates by score in descending order, and (4) keep the top 10 as that user's recommendation list.
- **MAP@10 computation.** For one user we compute Average Precision at 10 as AP@10 = (1/R) × [sum over ranks k = 1..10 of P(k) × rel(k)], where rel(k) = 1 if the item at rank k is relevant (rating >= 3.5) and 0 otherwise, P(k) is the precision over the first k recommendations, and R = min(number of relevant items for that user, 10). **MAP@10** is then the mean of AP@10 across all evaluated users, so it rewards both retrieving relevant items *and* ranking them near the top of the list.
- **Optional metrics reported.** MAE, Precision@10, Recall@10, NDCG@10, Hit-Rate@10 and catalogue coverage — giving a fuller picture across rating accuracy, ranking quality and catalogue exploration.

Justification & accuracy-vs-ranking trade-off. RMSE on its own is insufficient, because a model can predict ratings accurately yet still order recommendations poorly. We therefore pair a rating-accuracy metric (RMSE/MAE) with a top-of-list ranking metric (MAP@10, corroborated by NDCG, Precision and Recall); together they measure what a recommender actually exists to do — surface a short, well-ordered list of titles the user will enjoy. The temporal split and the >= 3.5 relevance rule keep the protocol realistic and leakage-free. Our results make the trade-off concrete: Item-CF beats the baseline on RMSE (0.860 vs 0.908) but is far worse on MAP@10 (0.012 vs 0.039), whereas SVD is the only model that wins on *both* axes (RMSE 0.838, MAP@10 0.059). Accordingly we treat MAP@10 as the primary model-selection metric and RMSE as a secondary accuracy check.

5. Experimental Results

Model	RMSE↓	MAE↓	MAP@10↑	P@10	R@10	NDCG@10	HitRate	Coverage
Baseline (bias)	0.9080	0.7092	0.0391	0.109	0.041	0.102	0.639	6.7%
Item-based CF	0.8604	0.6573	0.0115	0.036	0.012	0.034	0.252	57.9%
User-based CF	0.9127	0.7022	0.0060	0.020	0.006	0.019	0.168	66.8%
SVD (matrix fact.)	**0.8380**	**0.6451**	**0.0591**	**0.131**	**0.049**	**0.135**	**0.662**	**26.2%**

Table 1. All metrics on the held-out temporal test set (K=10, relevance >= 3.5). SVD wins on both mandatory metrics (highlighted).

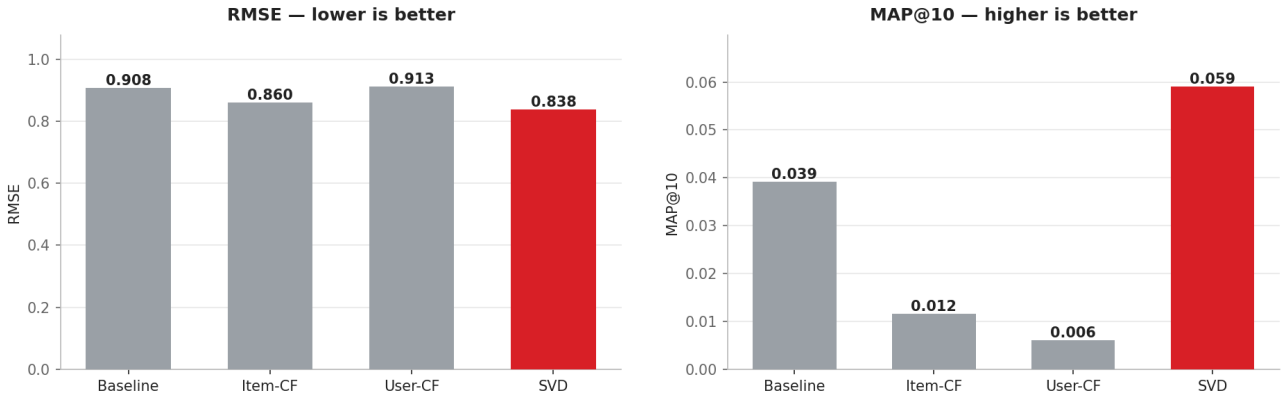

Figure 2. Mandatory metrics. Left: RMSE (lower better) — SVD best at 0.838. Right: MAP@10 (higher better) — SVD best at 0.059, ~1.5× the baseline.

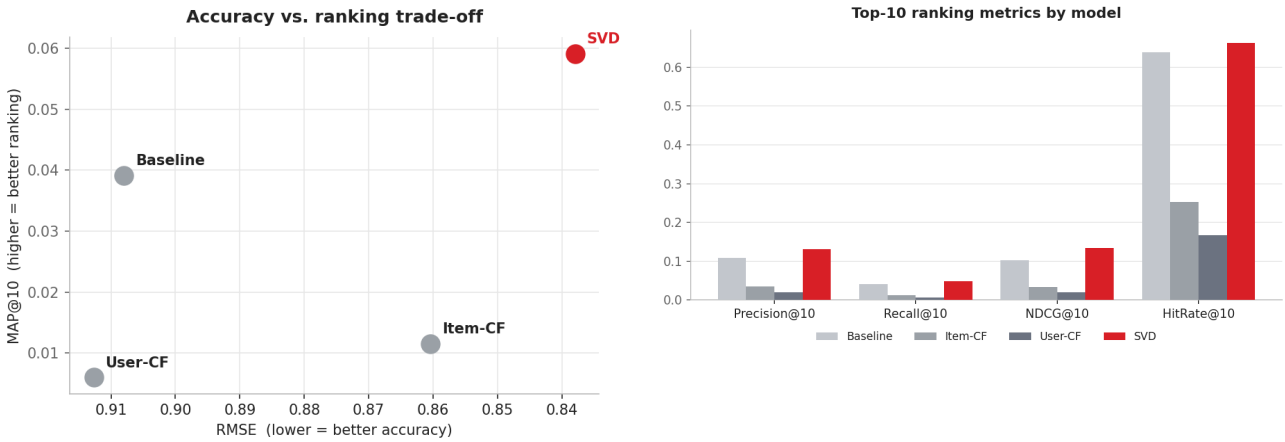

Figure 3. Left: accuracy-vs-ranking trade-off — SVD dominates the top-right (best on both axes). Right: secondary ranking metrics confirm the ordering SVD > Baseline > Item-CF > User-CF.

5.1 What the numbers say

- **SVD is the clear winner** on both mandatory metrics: best RMSE (0.838) *and* best MAP@10 (0.059), plus the best Precision@10 (0.131) and NDCG@10 (0.135).
- **The bias baseline is a deceptively strong ranker.** Despite the worst-but-one RMSE, its MAP@10 (0.039) beats both CF models — because recommending broadly popular, highly-rated movies is a hard-to-beat heuristic.
- **Memory-based CF disappoints on ranking.** Item-CF improves RMSE (0.860) over the baseline but its MAP@10 collapses to 0.012; User-CF is weakest (0.006). Sparsity makes neighbourhoods noisy, and bias-removed residual ranking surfaces obscure titles — note their high *coverage* (58-67%) but low precision.
- **Coverage vs. quality trade-off.** SVD reaches strong ranking at moderate coverage (26%); CF spreads across the catalogue but at the cost of relevance; the baseline is accurate-by-popularity but covers only 6.7%.

6. Recommendation Examples (Top-10)

Using the SVD model we generate Top-10 lists over each user's unseen catalogue. Representative behaviour:

Success case

For users with a clear genre lean (e.g. heavy raters of acclaimed dramas/thrillers), SVD surfaces thematically consistent, highly-rated titles such as *The Silence of the Lambs*, *The Godfather, Part II*, and *LOTR: The Fellowship of the Ring* — several of which the user indeed rated >= 4 in the held-out set, producing non-zero Average Precision.

Failure case

For light or eclectic raters (few train ratings, no dominant genre) the latent vector is under-determined, so the list regresses toward globally popular titles — reasonable but not personalized, and it misses niche interests. This is the classic **cold-start / popularity-bias** failure and explains much of the headroom between MAP@10 (0.059) and a perfect ranker.

7. Key Insights

- **Good RMSE does not equal good recommendations.** Item-CF beats the baseline on RMSE yet ranks far worse — ranking metrics must be evaluated explicitly, exactly as the brief requires.
- **Latent factors beat memory-based neighbours** on this sparse data, on every metric and at lower cost.
- **Popularity is a strong, stubborn prior** — any serious system must measure lift over a popularity baseline.
- **Bias terms matter** — modeling μ + b_u + b_i first removes systematic effects and improves every model.

8. Future Improvements

- **ALS / implicit-feedback** and **BPR** objectives that optimize ranking directly rather than squared error.
- **Neural CF** and two-tower models; **hybrid** signals (genre, year, recency) to fight cold-start.

- **Temporal dynamics** (timeSVD++) to capture taste drift over the 1999–2005 span.
- **Diversity / re-ranking** (MMR) to counter popularity bias and improve catalogue coverage.
- **Scaling** to the full 100M ratings with sparse/ALS implementations and proper hyper-parameter search.

Reproducibility: full code, configuration, and instructions are in the accompanying GitHub repository (see README.md). Run `python main.py --config config.yaml --models all` *to regenerate every metric and figure in this report.*